

NUMPY ARRAYS

- ```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2, 3)
np.zeros((2, 3)); np.ones(3); np.eye(3)
np.arange(0, 10, 2); np.linspace(0, 1, 5)
a.shape, a.ndim, a.dtype, a.size
a.reshape(3, 2); a.reshape(-1); a.T
a[0, 1]; a[:, 1]; a[a > 2] # index, column, mask
np.where(a > 2, 1, 0) # vectorised if/else
```
- One typed block of memory; operations are vectorised in C, avoid Python loops.
  - Basic slices are **views** (edit the original); fancy and boolean indexing return **copies**. `a.copy()` to be safe.
  - `np.random.default_rng(42).normal(size=5)`: always seed.
  - `ravel()` is a view when possible, `flatten()` always a copy; check with `np.shares_memory(a, b)`.
  - `a[:, None]` / `np.expand_dims` add an axis, `np.squeeze` drops length-1 axes. `np.stack` makes a new axis; `vstack/hstack/column_stack` reuse one. `np.split(a, 2)`.
  - `np.argwhere(a > 2)`: one [row, col] per hit. `np.isin(a, [...])`.

AXIS AND AGGREGATION

- `axis=0` collapses rows (one result per column); `axis=1` collapses columns (one per row).
  - `a.sum(axis=0)`, `.mean`, `.std(ddof=1)`, `.min/.max`, `.argmax`, `.cumsum`, `np.percentile(a, 90)`, `np.unique(a, return_counts=True)`.
  - `keepdims=True` keeps the axis so the result broadcasts back.
  - Sorting: `np.sort`, `np.argsort` (indices); `stack`: `np.concatenate`, `np.vstack`, `np.hstack`.
  - Linear algebra: `a @ b` (matmul), `np.dot`, `np.linalg.inv`, `.solve`, `.eig`, `.norm`.
  - Regression: `np.linalg.lstsq(X, y, rcond=None)[0]` with a ones column in X; safer than solving `X.T @ X`.
- TRAP** `a * b` is element-wise, `a @ b` is the matrix product: both run on square arrays. `np.std` divides by `n` (`ddof=0`); pandas `.std()` by `n - 1`. Pass `ddof` explicitly.

NUMPY EXTRAS

- Floats: `np.isclose(a, b)`, `np.nan` is a float; `np.nanmean`, `np.nansum`, `np.isnan(a)`; `np.inf`.
- `np.clip(a, lo, hi)`, `np.diff(a)`, `np.cumsum/cumprod`, `np.corrcoef(x, y)`, `np.histogram(a, bins=10)`, `np.searchsorted`.
- Dtypes: ints overflow silently (`int8`), `astype(float)`, `np.float32` halves memory. Mixed lists become `object`.
- Vectorise, then `np.vectorize` only as a convenience (it still loops). `np.select([c1, c2], [v1, v2], default)` for many branches.
- Random: `rng = np.random.default_rng(0)`; `rng.integers`, `rng.choice(a, size, replace=False)`, `rng.permutation`.

BROADCASTING RULES

Compare shapes from the **right**. Two dimensions are compatible if equal or one is 1 (the 1 is stretched). Missing leading dimensions count as 1.

| Shapes             | Result                                                 |
|--------------------|--------------------------------------------------------|
| (3, 4) with (4,)   | (3, 4) subtract column means                           |
| (3, 4) with (3, 1) | (3, 4) subtract row means (use <code>keepdims</code> ) |
| (3, 1) with (1, 4) | (3, 4) outer-style grid                                |
| (3, 4) with (3,)   | error                                                  |

Pairwise distances: `np.linalg.norm(x[:, None] - y[None], axis=2)`.

PANDAS: LOOK, SELECT, FILTER

- ```
import pandas as pd
df = pd.read_csv("f.csv", parse_dates=["d"])
df.shape; df.dtypes; df.head(); df.info()
df.describe(include="all"); df.isna().sum()
df["a"]; df[["a", "b"]] # Series / DataFrame
df.loc[rows, cols] # by label (end included)
df.iloc[0:3, 1:4] # by position (end excluded)
df[(df.a > 1) & (df.b == "x")] # & | ~ with brackets
df.query("a > 1 and b == 'x'")
df.loc[df.a > 1, "b"] = 0 # safe assignment
```
- Never chain `df[mask]["b"] = 0`: it may write to a copy (`SettingWithCopy`). Use `.loc`.
 - `df.assign(c=df.a * 2)`, `df.rename(columns=...)`, `df.drop(columns=...)`, `df.astype({"a": "int32"})`.

GROUPBY, AGG, TRANSFORM

- ```
df.groupby("g")["x"].mean()
df.groupby("g").agg(n=("x", "size"),
 avg=("x", "mean"))
df.groupby(["g", "h"], as_index=False).sum()
df["share"] = df.x / df.groupby("g")["x"].transform("sum")
df.groupby("g").head(2) # first rows per group
df.groupby("g")["x"].rank(ascending=False)
df.groupby("g")["x"].shift(1) # previous in group
df.groupby("g")["x"].cumsum()
```
- `agg` shrinks each group to one row; `transform` returns one value per original row (same index).
  - `size` counts rows, `count` counts non-NaN, `nunique` distinct.

COMBINE AND RESHAPE

- ```
pd.merge(a, b, on="id", how="left")
a.merge(b, left_on="x", right_on="y")
pd.concat([a, b], axis=0, ignore_index=True)
a.join(b, on="k") # join on index
df.pivot_table(index="r", columns="c",
               values="v", aggfunc="sum")
df.melt(id_vars="id", var_name="k", value_name="v")
df.explode("tags"); df.stack(); df.unstack()
```
- `merge(..., validate="one_to_one")` catches key duplicates; `indicator=True` adds `_merge` (anti-join: `left_only`).
 - `how`: `inner` (default), `left`, `right`, `outer`.

MISSING, DUPLICATES, TYPES

- ```
df.isna().mean() # share missing
df.dropna(subset=["a"]); df.dropna(how="all")
df["a"].fillna(df["a"].median())
df.fillna({"a": 0, "b": "NA"}); df.ffill()
df.duplicated(subset=["id"]).sum()
df.drop_duplicates("id", keep="last")
pd.to_datetime(df.d); pd.to_numeric(s, errors="coerce")
df["c"] = df["c"].astype("category")
```
- `NaN`  $\neq$  `NaN`; use `isna()`. Mean skips NaN by default (`skipna=True`).
  - Sort before `drop_duplicates(keep="last")` to keep the latest.
  - Memory: `category` for low-cardinality strings, `downcast` numerics, read only needed columns (`usecols`), `chunks`.

STRINGS AND DATES

- ```
df.s.str.lower().str.strip().str.contains("x")
df.s.str.split(",", expand=True); .str.extract(r"(\d+)")
df.d.dt.year; .dt.month; .dt.dayofweek; .dt.floor("D")
df.set_index("d").resample("W").sum()
df.x.rolling(7).mean(); df.x.expanding().max()
df.x.pct_change(); df.x.diff(); df.x.shift(7)
pd.cut(df.x, bins=[0, 18, 65, 120]) # fixed bins
pd.qcut(df.x, 4) # quantile bins
```

SQL -> PANDAS

SQL	pandas
WHERE	boolean mask / query
SELECT a, b	df[["a", "b"]]
GROUP BY ... AVG	groupby().agg()
HAVING	filter the grouped result
JOIN	merge(how=...)
ORDER BY ... LIMIT	sort_values().head()
COUNT(DISTINCT x)	nunique()
ROW_NUMBER	groupby().cumcount() + 1
RANK	groupby().rank(method="min")
LAG	groupby().shift(1)
SUM() OVER	groupby().transform("sum")
CASE WHEN	np.where / np.select
UNION ALL	pd.concat
DISTINCT	drop_duplicates()
anti-join	merge(indicator=True) then left_only

WORKED PATTERNS

```
# top 2 per group
df.sort_values("x", ascending=False).groupby("g").head(2)
# latest row per user
df.sort_values("ts").drop_duplicates("u", keep="last")
# conversion by group
df.groupby("g").agg(rate=("conv", "mean"))
# streak id: new id when the value changes
s = (df.v != df.v.shift()).cumsum()
# cohort retention table
df["m"] = df.d.dt.to_period("M")
first = df.groupby("u").m.transform("min")
df["age"] = (df.m - first).apply(lambda p: p.n)
df.pivot_table(index=first, columns="age",
                values="u", aggfunc="nunique")
```

State the output grain first; build in steps and print after each.

INDEX, APPLY, WINDOW HELPERS

- df.set_index("k"), reset_index(), sort_index(); label slicing needs a sorted index. df.loc["a":"c"] includes "c".
- Series.map({...}) / .map(func) elementwise; df.apply(func, axis=0/1) per column / row; applymap/DataFrame.map per cell.
- df.nlargest(3, "x"), df.sample(frac=.2, random_state=0), df.clip(lower, upper), df.where(cond, other).
- Correlation: df.corr(numeric_only=True) (Pearson), df.corr(method="spearman"), df.cov().
- Counts: s.value_counts(normalize=True), pd.crosstab(a, b), s.nunique().
- Split data: from sklearn.model_selection import train_test_split; train_test_split(X, y, test_size=.2, stratify=y, random_state=0).
- Display: pd.set_option("display.max_columns", 50); save with df.to_csv("o.csv", index=False), to_parquet.

CLEAN, ENCODE, RESHAPE: EXTRAS

```
pd.to_datetime(s, format="%Y-%m-%d", errors="coerce")
s.str.extract(r"([A-Z]+)-(\d+)") # groups → columns
s.str.contains("x", case=False, na=False)
s.dt.tz_localize("UTC").dt.tz_convert("Asia/Kolkata")
pd.get_dummies(df, columns=["c"], drop_first=True)
s.ffill(); s.bfill(); s.interpolate() # ordered data
df.dropna(axis=1, thresh=int(.5 * len(df)))
df.reindex(labels) # missing labels → NaN
df.query("a > @limit") # @ reads a variable
pd.read_sql(query, conn); df.to_sql("t", conn)
pd.crosstab(a, b, normalize="index", margins=True)
s.ewm(span=7).mean(); s.cummax(); s.shift(-1) # lead
df.pipe(func, arg) # own step in a chain
df.memory_usage(deep=True).sum()
```

- errors="coerce" turns bad values into NaN / NaT: count them afterwards. astype fails on the first bad value.
- pivot only reshapes (duplicate pairs raise); pivot_table aggregates; melt goes wide to long.
- Stacked series: groupby("g")["x"].shift(1), never a plain shift. A lead (shift(-1)) is a target, not a feature: leakage. bfill and interpolate also look ahead.
- Speed, fastest first: NumPy arrays, pandas vectorised, np.where/np.select, comprehension over zip(df.a, df.b), itertuples, apply(axis=1), iterrows (thousands of times slower).
- category pays when distinct values are far fewer than rows (a 200k-row table: 24.5 MB to 2.1 MB).

SPEED AND GOTCHAS

- Prefer vectorised column ops; apply(axis=1) is a Python loop (slow). np.where, .str, .dt, map(dict) are faster.
 - iterrows is slow; itertuples is faster; vectorise if you can.
 - inplace=True rarely saves memory; assign back instead.
 - = on a column of strings with NaN gives False for NaN. df.a.isin([...]) for membership.
 - df.copy() before modifying a slice you keep.
- TRAP** After a merge, compare row counts: duplicate keys fan out rows. groupby drops NaN keys by default (dropna=False to keep). Integer columns with NaN become float.

CHART CHOICE

Question	Chart
Compare categories	bar (start at zero)
Trend over time	line
Distribution of one variable	histogram, box, violin
Relationship of two numbers	scatter (+ trend line)
Many numeric pairs	correlation heatmap, pair plot
Part of a whole	stacked bar (pie only for 2-3 parts)
Ranking	sorted horizontal bar

```
import matplotlib.pyplot as plt, seaborn as sns
fig, ax = plt.subplots(figsize=(6, 3))
sns.histplot(df.x, bins=30, ax=ax)
sns.boxplot(data=df, x="g", y="v")
sns.heatmap(df.corr(numeric_only=True), annot=True)
ax.set(title="Takeaway as title", xlabel="x (unit)")

fig, axs = plt.subplots(1, 2, figsize=(8, 3))
sns.scatterplot(data=df, x="a", y="b", hue="g")
ax.set_yscale("log"); ax.tick_params(axis="x", rotation=45)
fig.tight_layout(); fig.savefig("f.png", dpi=150)
```

Label axes with units, one colour unless colour means something, avoid dual axes and 3-D, put the takeaway in the title, log scale for skewed data.

EDA ROUTINE

- Shape, dtypes, a few rows, memory.
- Missing values by column and by pattern: why missing?
- Duplicates and impossible values (negative age, future date).
- Each variable's distribution: skew, outliers, rare categories.
- Target versus each feature; correlations (leads, not proof).
- Segments and time trends; check data drift.
- Write findings and next steps. Decide on training data only.

SAY IT IN 20 SECONDS

- "transform keeps the rows, agg collapses them."
- "Broadcasting aligns shapes from the right; 1 stretches."
- "loc is label-based and inclusive, iloc positional and exclusive."
- "Slices are views, boolean masks are copies."